

Селекторы. Виды селекторов и основные CSS свойства

Селекторы — это часть правила CSS, которые позволяют определить к какому элементу веб страницы следует применять правила. Они могут быть: простыми, сложными, составными (список селекторов).

К простым селекторам относятся:

- классы;
- селектор тега;
- идентификаторы;
- универсальный селектор.

Простые селекторы

Классы задаются элементам с помощью атрибута `class=»имя_класса»`.

Например:

`<div class=»content»>` Блок с основным контентом `</div>`

Главная и отличительная особенность классов в том, что один класс может быть присвоен нескольким тегам одновременно. Соответственно правила CSS будут применяться ко все элементам содержащим атрибут `class` с соответствующим значением.

Описание классов CSS:

Все классы на CSS записываются в следующем виде:

```
.content {  
width: 50%;  
color: green;  
}
```

Теперь после того как мы присвоим элементу класс со значением контент, он будет иметь ширину 50% и цвет текста зеленый.

`<div class=»content»>`Ширина 50% и цвет текста зеленый`</div>`

Как мы и говорили ранее классы можно назначать многим элементам веб-страницы. То есть, если у вас на одной странице будет несколько элементов с одним классом, то ошибкой это не будет.

Селекторы тегов правила написанные для селектора тега будут применяться ко всем одинаковым тегам на веб странице. В качестве самого селектора выступают названия тегов.

Например:

```
p {  
color: green;
```

}

В данном случае ко всем тегам <P> будет применен стиль написанный выше. При таком написании правил не нужно добавлять дополнительные атрибуты. Правила применяются ко всем тегам имеющим заданное название.

Следует учитывать факт применения ко всем тегам правил. В основном данные селекторы используются для того чтобы задать всем тегам одни и те же правила. В основном это правила определяющие написание текста (шрифт, размер, цвет и тп.).

Идентификаторы в отличии от классов могут применяться только к одному объекту веб страницы. Присутствие на странице двух объектов с одним идентификатором не допустимо. Это вызовет ошибку компиляции кода. Присваивается идентификатор тегу с помощью атрибута id.

Вид идентификатора в HTML

<p id=»content_plus»>Второго такого идентификатора быть не должно</p>

В коде CSS идентификаторы описываются следующим образом:

```
#content_plus {  
    color: red;  
}
```

Идентификаторы используются довольно редко для описания стилей. В основном атрибут id для тега предназначен для работы со скриптами. Особенно активно используется в [JavaScript](#).

Составные селекторы

К составным селекторам относятся простые селекторы, перечисленные через запятую. Применяются когда к нескольким объектам с различными классами необходимо применить один стиль оформления. Конечно можно изменить написание каждого правила, но это займет время и увеличит размер CSS. Следовательно увеличится время загрузки страницы.

Пример составного селектора:

<div class=»content»> блок контента сайта </div>

<p class=»foot»> блок подвала сайта </p>

Код CSS выглядит следующим образом:

```
.content {  
    color: red;  
}  
.foot {  
    color: green;  
}
```

```
.content, .foot {  
width: 100%;  
}
```

В данном случае у нас есть два разных элемента с разными классами. Каждый класс содержит определенный стиль оформления. Это цвет текста в основном блоке цвет красный, в подвале зеленый. появилась необходимость сделать эти элементы шириной 100%. Можно дописать еще по строчке к каждому классу, но как уже сказали, это увеличит объем работы и вес файла CSS. В нашем случае код очень маленький поэтому эта разница не особо заметна. Но представьте что у вас есть 1000 элементов, сотни классов и идентификаторов, 2000 строк кода. И вам потребовалось некоторые элементы изменить, например убрать или добавить отступ. Таких элементов может быть 20, 30 и больше. И тогда вам придется добавить не 2 строки а 20 или 30 строк. Кроме этого вам нужно найти все места где описаны нужные селекторы а это занимает время. В этом случае составные селекторы очень облегчают задачу верстальщику и сокращают время загрузки страницы.

Есть и еще вид составных селекторов. Например, необходимо добавить стиль оформления для определенных тегов содержащих определенный класс. Как говорили ранее, классы могут применены ко многим элементам веб страницы.

```
div.content {  
margin-top: 5 px;  
}
```

Данный стиль оформления будет применен к тегам div с классом content.

<div class=»content» > стиль будет применен **</div>**

<p class = «content»> Стиль применен не будет**</p>**

Сложные селекторы

В основном это несколько селекторов которые разделены комбинаторами. Комбинаторов может быть несколько: вложенности, дочерние, последовательные.

Например:

```
div p {  
color: green;  
}
```

Данный код CSS будет применен ко всем тегам <P> которые находятся внутри тега <DIV>

<div> <p> Стиль будет применен **</p></div>**

<p> <div> Стиль применен не будет **</div> </p>**

Это селектор вложенности. Вложенность может быть применена не только к селекторам тегов. Так же она работает и с классами, идентификаторами и тп.

Другой пример:

```
div.content p.blok {  
width: 30 px;  
}
```

В данном примере стиль будет применен для тега `<p>` которому присвоен класс `blok`, который в свою очередь находится в теге `<div>` которому присвоен класс `content`.

```
<div class=»content»> <p class=»blok»>Стиль будет применен</p></div>  
<div class=»foot»> <p class=»blok»>Стиль не будет применен</p></div>
```

Далее идет селектор на дочерние элементы. Данный селектор применяет стиль оформления строго к дочерним элементам. Очень похоже на вложенность только с одним отличием. Селектор вложенности применяет стиль ко всем вложенным элементам а этот строго к дочерним, исключая другие.

Например:

```
div>p {  
color: yellow;  
}
```

Данный стиль будет применяться к тегам `<p>` которые находятся только в теге `<div>`.

```
<div> <p> Стиль будет применен</p> </div>
```

```
<div> <span> <p> Стиль не будет применен так как p — это дочерний  
элемент тега span </p></span></div>
```

И последние сложные селекторы это последовательные. Они применяют стили к элементам стоящим после каких либо других.

Например:

```
div+p {  
width: 30px;  
}
```

Стиль будет применен к тегу `<p>` идущему после тега `<div>`

```
<div> контент </div> <p>Стиль будет применен</p>
```

```
<div> контент </div><span>Информация</span> <p>Стиль не будет  
применен</p>
```

Рассмотрим на примерах:

каждому тегу в HTML соответствует так называемый **селектор** CSS. К примеру, тегу `<p>` соответствует CSS селектор `p`, с помощью которого мы можем обратиться ко всем абзацам HTML страницы и, например, покрасить их всех одновременно в красный

цвет. После селектора следует ставить фигурные скобки {}, внутри которых следует писать **CSS свойства**.

CSS свойства и задают цвет, размер шрифта и другие интересные вещи. Их следует писать в таком формате: **имя** свойства, потом **двоеточие**, потом **значение** этого свойства (например, свойство - это цвет, а "красный" - это значение). Потом нужно поставить **точку с запятой** и можно писать следующее свойство.

Давайте покрасим все абзацы в красный цвет:

Это HTML код:

```
<p> Абзац с текстом. </p>
```

Это CSS код:

```
p {  
    color: red;  
}
```

Так код будет выглядеть в браузере:

Абзац с текстом.

А теперь давайте покрасим абзацы в **зеленый** цвет и сделаем их **большого** размера:

```
<p>
```

Абзац с текстом.

```
</p>
```

```
p {  
    color: red;  
    font-size: 25px;  
}
```

Так код будет выглядеть в браузере:

Абзац с текстом.

Обратите внимание на оформление CSS кода: каждое новое свойство пишется с новой строки.

Еще раз рассмотрим коротко подключение css к вашему html документу(для тех кто забыл или не понял)

В следующем примере к нашему HTML файлу подключается CSS файл **style.css**:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset="utf-8">
```

```

<title>Это заголовок тайтл</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
  <p>
    Это абзац с текстом.
  </p>
</body>
</html>

```

Можно подключать не один файл, а несколько. Давайте подключим файлы style1.css, style2.css и style3.css:

```

<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Это заголовок тайтл</title>
    <link rel="stylesheet" href="style1.css">
    <link rel="stylesheet" href="style2.css">
    <link rel="stylesheet" href="style3.css">
  </head>
  <body>
    <p>
      Это абзац с текстом.
    </p>
  </body>
</html>

```

Способ 2. CSS внутри тега style

Второй способ заключается в том, что CSS код можно написать в теге **<style>** Его можно размещать как внутри **<head>** (чаще всего), так и внутри **<body>**.

```

<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Это заголовок тайтл </title>
    <style>
      p {
        color: red;
      }
    </style>
  </head>
  <body>
    <p>
      Это абзац с текстом.
    </p>
  </body>
</html>

```

```
        }  
    </style>  
</head>  
<body>  
    <p>  
        Это абзац с текстом.  
    </p>  
</body>  
</html>
```

Недостаток этого способа в том, что CSS код применяется только к одной странице сайта, а не ко многим. Этот способ обычно применяется тогда, когда нужно написать CSS на конкретной странице сайта, не затрагивая остальных, и, в общем случае, к употреблению не рекомендуется.

Блок Комментарии CSS

Так же, как и в **HTML**, в **CSS** можно ставить **комментарии**. Они оформляются следующим образом: **слеш и звездочка** */**, потом *текст комментария*, потом **звездочка и слеш** **/*. Смотрите пример:

```
p {  
    /*  
        Тут находится  
        комментарий CSS.  
    */  
  
    color: red;  
    font-size: 27px;  
}
```

Основные CSS свойства

*Мы с вами разобрались с различными способами подключения CSS на HTML страницу в предыдущем уроке и теперь давайте начнем изучать различные **CSS свойства**.*

Блок 1. Свойство color - цвет текста

Свойство **color** позволяет задать **цвет текста**. Цвет можно задавать следующими способами: английским **словом**, через решетку **#**, через **rgb** и некоторыми другими способами, которые мы пока не будем разбирать.

Давайте разберем указанные способы подробнее.

Способ первый - английское слово

Самый простой и понятный вариант задать цвет - указать его английским словом, например, **red** задает *красный* цвет, **blue** - *голубой*, **green** - *зеленый*, **black** - *черный*, **white** - *белый* (все названия цветов, которые можно использовать, смотрите тут: [цвета CSS](#)).

В следующем примере всем абзацам на странице задан красный цвет:

```
<p>
```

Абзац с текстом.

```
</p>
```

```
p {
```

```
color: red;
```

```
}
```

Так код будет выглядеть в браузере:

Абзац с текстом.

Первый способ задания цвета имеет некоторые недостатки. **Во-первых**, таким образом можно сделать не все оттенки цветов (все-таки количество названий ограничено, а оттенков можно придумать огромное множество). **Во-вторых**, невозможно помнить все названия цветов наизусть и, если нужен какой-то редкий цвет, приходится лазить по справочнику и искать его.

Поэтому в CSS существуют и более универсальные способы задать нужный вам цвет. Чтобы понять эти способы, для начала вам необходимо разобраться с тем, как получается нужный цвет на экране компьютера.

На самом деле отдельная точка экрана (*пиксель*) не может светиться всеми цветами, которые нужны, так как это технически было бы очень сложно. И на самом деле каждая точка экрана может светиться только тремя цветами: красным, зеленым и голубым. Причем одновременно и в разных пропорциях.

Комбинируя эти цвета мы можем получить любой нужный нам цвет подобно тому, как это делают художники с красками - если смешать два цвета - мы получим третий.

В CSS есть способ заставить пиксели светиться указанными тремя цветами в нужных нам пропорциях. Давайте его разберем:

Способ второй - через rgb

Суть этого способа заключается в следующем - для свойства **color** я пишу следующее значение - *rgb(красный, зеленый, голубой)* - и указываю в каких пропорциях нужно брать эти три базовых цвета. Сами цвета могут изменяться от **0** до **255**. Причем ноль - это отсутствие цвета, а 255 - это чистый цвет (к примеру, чисто красный).

Сами

буквы **rgb** расшифровываются

как *red* (красный), *green* (зеленый), *blue* (голубой).

Давайте смешаем чисто красный и чисто зеленый цвет. Для этого первое значение ставим в **255**, второе тоже в **255**, а третье - в **ноль**, так как примесь голубого нам не

нужна. У нас получится чисто желтый цвет (я поставил голубой фон блоку, так как желтый цвет на белом режет глаза):

```
<p>
    Абзац с текстом.
</p>
p {
    color: rgb(255, 255, 0);
}
```

Так код будет выглядеть в браузере:

Абзац с текстом.

А вот так мы получим чисто **красный** цвет (первое значение в 255, а все остальные нули):

```
<p>
    Абзац с текстом.
</p>
p {
    color: rgb(255, 0, 0);
}
```

Так код будет выглядеть в браузере:

Абзац с текстом.

Так получим чисто **зеленый** цвет:

```
<p>
    Абзац с текстом.
</p>
p {
    color: rgb(0, 255, 0);
}
```

Так код будет выглядеть в браузере:

Абзац с текстом.

А вот так - **частично зеленый** (я поставил не 255, а 100 для зеленого):

```
<p>
    Абзац с текстом.
</p>
p {
    color: rgb(0, 100, 0);
}
```

Так код будет выглядеть в браузере:

Абзац с текстом.

Ну, а теперь я смешаю все три цвета в **произвольных** пропорциях. Смотрите, что получилось:

```
<p>
```

Абзац с текстом.

```
</p>
```

```
p {
```

```
    color: rgb(200, 100, 125);
```

```
}
```

Так код будет выглядеть в браузере:

Абзац с текстом.

Если я поставлю все значения в 255, то получу чистый белый цвет, а если все нули - чистый черный.

Чисто **белый** цвет:

```
<p>
```

Абзац с текстом.

```
</p>
```

```
p {
```

```
    color: rgb(255, 255, 255);
```

```
}
```

Так код будет выглядеть в браузере:

Абзац с текстом.

Чисто **черный** цвет:

```
<p>
```

Абзац с текстом.

```
</p>
```

```
p {
```

```
    color: rgb(0, 0, 0);
```

```
}
```

Так код будет выглядеть в браузере:

Абзац с текстом.

Принцип смешения цветов нужен вам только для понимания происходящих процессов, не стоит его сильно пугаться, если вы его не совсем поняли. Существуют специальные **программы**, которые позволяют выбрать любой оттенок любого цвета и получить его rgb. Вы будете только копировать нужные вам значения из программы и спокойно пользоваться, не задумываясь о смешении цветов. Об этом я расскажу вам в видео после того, как объясню третий способ работы с цветом.

Способ третий - через

Второй способ получения цвета (через rgb), не смотря на то, что позволяет получить любой оттенок любого цвета, все же является несколько громоздким. Поэтому существует и третий вариант задать цвет - через **шестнадцатеричное** значение.

Чтобы понять суть этого способа, вам нужно разобраться с **шестнадцатеричной** системой счисления. В ней, в отличие от десятичной, которой мы с вами пользуемся не 10 цифр, а 16: **0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F** - как вы видите, за недостатком цифр после девяти **используются буквы**. Преимущество шестнадцатеричной системы в том, что число 255 (и менее) можно представить всего двумя символами - **FF** соответствует **255** и так далее.

В этой системе вместо **rgb()** записывается решетка **#**, после которой идут **6** знаков. Первые 2 знака - красный цвет, вторые два знака - зеленый, и последние два знака - голубой. Цвета также изменяются от **0** до **255**, но в шестнадцатеричной системе это будет от **00** до **FF**.

К примеру, запись **rgb(255, 200, 255)** можно представить как **#FFC8FF** - это будет более компактно.

В следующем примере абзац красится в красный цвет:

`<p>`

Абзац с текстом.

`</p>`

`p {`

`color: #FF0000;`

`}`

Так код будет выглядеть в браузере:

Абзац с текстом.

Мы уже разбирали, что **rgb(0, 0, 0)** соответствует черному цвету, а значит **#000000** - тоже черный цвет, а **rgb(255, 255, 255)** - белый и **#FFFFFF** - тоже белый.

Кроме того, существует сокращенная форма записи - к примеру, **#FEDFED** можно переписать как **#FED** - то есть, если первая половина записи совпадает со второй половиной - то ее (вторую половину) можно не писать. В сокращенном варианте черный цвет будет **#000**, а белый - **#FFF**.

Опять-таки, если вы чего-то не понимаете в этих заумных рассуждениях - ничего страшного, потому что в реальной жизни для этого все пользуются **специальными программами** для определения цвета и просто копируют оттуда цвет в нужном формате. Просто запомните, что есть **три способа** работы с цветом (на самом деле их больше, о других будет намного позже) и научитесь с ними работать. А про смешение цветов можно особо и не понимать - это вам для общего развития, чтобы вы имели представление о том, откуда растут ноги всего этого.

Блок .2 Свойства width и height - ширина и высота

Свойства **width** и **height** позволяют задать высоту и ширину элементу соответственно. Ширина и высота обычно измеряются в **пикселях** (обозначается **px**) или **процентах** (обозначается **%**) (можно измерять и других единицах, которые мы пока не будем разбирать).

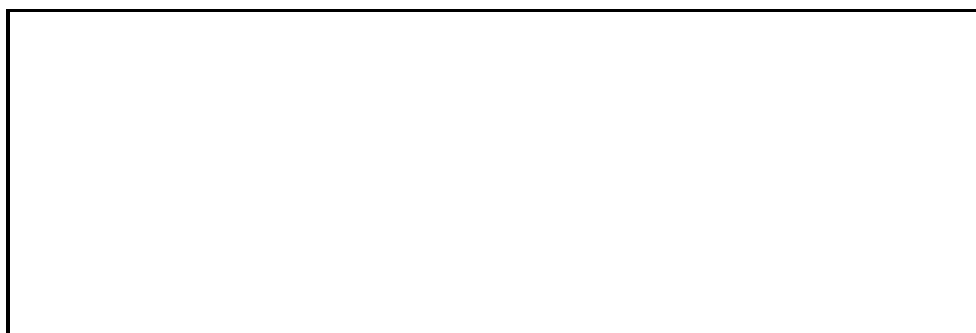
Пиксель - это минимальная точка на экране. При некотором опыте вы сможете легко определять на глаз, сколько пикселей размер у того или иного элемента. Для этого также можно использовать и специальные измерительные инструменты.

Если задавать размеры в **процентах** - то эти проценты вычисляются **относительно родительского элемента**.

В данном примере абзацу задана ширина и высота, а также **граница**, чтобы можно было увидеть, где заканчивается наш абзац (границу мы еще не проходили, пока не обращайте на нее внимание, чуть позже вы узнаете, как ее сделать):

```
<p>
    Абзац с текстом.
</p>
p {
    width: 300px;
    height: 100px;
    border: 1px solid black;
}
```

Так код будет выглядеть в браузере:



Блок . 3 Свойство text-align - выравнивание текста

Свойство **text-align** позволяет задать выравнивание текста. Текст можно выравнивать по **левому** краю (значение **left**), по **правому** (значение **right**), по **центру** (значение **center**) и **одновременно** и по правому, и по левому краю (значение **justify**).

Давайте посмотрим на примерах, что имеется в виду.

Значение left

Давайте сделаем так, чтобы текст был выровнен по **левому** краю. Для этого нужно свойство **text-align** поставить в значение **left**:

```
<p>
    Абзац с текстом.
</p>
p {
    text-align: left;
}
```

```
width: 300px;
}
```

Так код будет выглядеть в браузере:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velit sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehicula, ante purus lacinia dui, interdum fringilla massa eros ut dui.

В принципе, для абзацев значение **left** можно и не ставить - они по умолчанию и так выровнены по **левому** краю. Однако есть элементы, которые по умолчанию стоят **по центру** (это, например, теги **th**, которые делают ячейку-заголовок таблицы). И вот для них как раз-таки и может потребоваться выравнивание по левой стороне.

Значение right

Давайте теперь поставим текст по **правому** краю:

```
<p>
    Абзац с текстом.
</p>
p {
    text-align: right;
    width: 300px;
}
```

Так код будет выглядеть в браузере:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velit sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehicula, ante purus lacinia dui, interdum fringilla massa eros ut dui.

Значение center

Поставим текст **по центру**:

```
<p>
    Абзац с текстом.
</p>
p {
    text-align: center;
    width: 300px;
}
```

Так код будет выглядеть в браузере:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velit sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehicula, ante purus lacinia dui, interdum fringilla massa eros ut dui.

Значение justify

Ну, и наконец выровняем текст одновременно и по правому, и по левому краю:

```
<p>
```

Абзац с текстом.

```
</p>
```

```
p {
```

```
text-align: justify;
```

```
width: 300px;
```

```
}
```

Так код будет выглядеть в браузере:

Lorem ipsum dolor sit amet, consectetur
adipiscing elit. Aenean a dapibus magna, ac
interdum nisl. Suspendisse eget fringilla nibh,
eu commodo arcu. Donec lacinia tempor velit
sed tincidunt. Aliquam porttitor nulla purus,
vel vulputate ipsum faucibus sed. Phasellus
sodales, lorem vel cursus vehicula, ante purus
lacinia dui, interdum fringilla massa eros ut
dui.

Блок 4. Свойство font-weight - жирность

Свойство **font-weight** позволяет сделать текст жирным или наоборот - отменить жирность (к примеру, для заголовков, которые жирные по умолчанию). Чтобы сделать текст жирным, следует добавить значение **bold**, а чтобы отменить жирность - значение **normal**.

Это свойство работает аналогично [text-align](#), который мы с вами уже разобрали. Разница в том, что через CSS управлять жирностью гораздо удобнее - я могу заставить все абзацы стать жирными, а все заголовки - нежирными, сделав всего пару записей в CSS файле.

Если бы я делал жирный через **тег ** - мне пришлось бы содержимое каждого абзаца обернуть в этот тег, а это было бы очень затратным по времени (а если я потом захочу убрать жирность абзацам - мне придется убирать все эти теги, представьте, сколько это лишней, бесполезной работы).

В следующем примере мы сделаем все абзацы жирными, а все заголовки h3 - нежирными:

```
<h3>Это заголовок</h3>
```

```
<p>
```

Абзац с текстом.

```
</p>
```

```
h3 {
```

```
    font-weight: normal;
```

```
}
```

```
p {
```

```
    font-weight: bold;
```

```
}
```

Так код будет выглядеть в браузере:

Это заголовок

Абзац с текстом.

Для сравнения привожу их вид по умолчанию:

Это заголовок

Абзац с текстом.

Блок 5. Свойство font-style - курсив

Свойство **font-style** позволяет сделать текст курсивным или наоборот - отменить курсив. Чтобы сделать текст курсивом, следует добавить значение **italic**, а чтобы отменить курсив - значение **normal**. Смотрите пример:

```
<p>
```

Абзац с текстом.

```
</p>
```

```
p {
```

```
    font-style: italic;
```

```
}
```

Так код будет выглядеть в браузере:

Абзац с текстом.

Блок 6 . Свойство font-size - размер текста

Свойство **font-size** позволяет задать размер текста. Размер задается в **пикселях** (обозначаются **px**), в **пунктах** (обозначаются **pt**), в **процентах** (обозначаются **%**) и с помощью некоторых других единиц, которые мы пока не будем разбирать.

С **пикселями** вы уже знакомы и даже знаете измерительные инструменты для этого. Давайте разберемся с тем, что такое пункты.

Значение в пунктах

Пункты можно понять, если вы работали с программой Word. Там вы можете задать, к примеру, шрифт размером 16 - это число и есть размер в пунктах. При этом этот шрифт в Ворде будет такого же такого же размера, как и шрифт 16 пунктов в браузере.

Между пунктами и пикселями есть соответствие: **12pt = 16px**. При этом все размеры на экране на самом деле меряются в пикселях, даже если вы задаете их в пунктах. При этом, если после перевода пунктов в пиксели, полученные пиксели будут дробными - они округлятся до ближайшего целого.

Давайте зададим тексту абзаца шрифт в 20 пунктов:

```
<p>
```

Абзац с текстом.

```
</p>
```

```
p {
```

```
font-size: 20pt;
```

```
}
```

Так код будет выглядеть в браузере:

Абзац с текстом.

Единственное удобство пунктов в том, что размеры, заданные в них, будут такими же, как и в программе Word. Если вам это удобно - пользуйтесь ими, если нет - можно спокойно пользоваться пикселями.

Значение в процентах

Значение размера шрифта в процентах мы пока не будем разбирать, просто знайте, что такое есть. При необходимости смотрите страницу в справочнике CSS

Блок7. Свойство font-family - тип шрифта

Свойство **font-family** позволяет задать тип шрифта (тип часто называют *семейством* шрифта).

Поставим для всех абзацев шрифт **Arial**:

```
<p>
```

Абзац с текстом.

```
</p>
```

```
p {
```

```
font-family: Arial;
```

```
}
```

Так код будет выглядеть в браузере:

Абзац с текстом.

Свойство line-height - межстрочный интервал

Свойство **line-height** задает межстрочный интервал.

Межстрочный интервал - это расстояние между линиями текста, то есть белый промежуток между ними.

При использовании свойства **line-height** вас может ожидать некоторый **подвох**: это свойство не задает промежуток между строками текста, как могло бы показаться, а задает **высоту линии текста**.

Это значит, что реальный видимый промежуток между строками будет вычисляться так: **line-height - font-size = расстояние между строками текста**. Или наоборот **line-height = font-size + расстояние между строками текста**.

В данном примере расстояние между строками текста будет **line-height - font-size = 50px - 13px = 37px**:

```
<p>
```

Длинный текст...

```
</p>
```

```
p {
```

```
    font-size: 13px;
```

```
    line-height: 50px;
```

```
}
```

Так код будет выглядеть в браузере:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velit sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehicula, ante purus lacinia dui, interdum fringilla massa eros ut dui.

Блок 8. Свойство font - сокращение для шрифтов

Существует специальное свойство **font**, которое можно использовать вместо многих свойств, которые мы уже разобрали. Такие свойства в CSS называются **свойствами-сокращениями**. Зачастую их использование гораздо удобнее вместо множества других свойств.

Свойство **font** имеет следующий синтаксис: **курсив** **жирность** **размер_шрифта/интервал_между_строками** **семейство_шрифта**. Обязательными являются "размер_шрифта" и "семейство_шрифта", порядок имеет значение.

Свойство **font** является **свойством-сокращением**.

```
<p>
```

Длинный текст...

```
</p>
```

```
p {
```

```
font: 16px Arial;
```

```
}
```

Так код будет выглядеть в браузере:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velit sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehicula, ante purus lacinia dui, interdum fringilla massa eros ut dui.

```
<p>
```

Длинный текст...

```
</p>
```

```
p {
```

```
font: italic 16px/50px Arial, sans-serif;
```

```
}
```

Так код будет выглядеть в браузере:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velit sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehicula, ante purus lacinia dui, interdum fringilla massa eros ut dui.

Блок 9. Свойство text-indent - красная строка

Свойство **text-indent** позволяет задать **красную строку**, то есть отступ первой строки текста (к примеру, в абзаце).

Давайте сделаем красную строку для абзацев:

```
<p>
```

```
        Длинный текст...
    </p>

    <p>

        Длинный текст...

    </p>

    <p>

        Длинный текст...

    </p>
p {
    text-indent: 50px;
}
```

Так код будет выглядеть в браузере:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velit sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehicula, ante purus lacinia dui, interdum fringilla massa eros ut dui.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velit sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehicula, ante purus lacinia dui, interdum fringilla massa eros ut dui.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velit sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehicula, ante purus lacinia dui, interdum fringilla massa eros ut dui.

Блок 10. Свойство vertical-align

Свойство **vertical-align** позволяет задать выравнивание по вертикали. К сожалению, адекватным образом это выравнивание работает только для таблиц.

Значение **top** выравнивает по верхнему краю, значение **bottom** - по нижнему, а **middle** - по центру по вертикали (есть еще некоторые другие значения, они нас пока не интересуют).

Значение top

Давайте сделаем так, чтобы по вертикали текст в ячейках таблицы был выровнен **по верхнему краю**:

```
td {
```

```

        vertical-align: top;
    }
<table border="1" height="200">
    <tr>
        <td>Ячейка 1</td>
        <td>Ячейка 2</td>
        <td>Ячейка 3</td>
    </tr>
    <tr>
        <td>Ячейка 4</td>
        <td>Ячейка 5</td>
        <td>Ячейка 6</td>
    </tr>
    <tr>
        <td>Ячейка 7</td>
        <td>Ячейка 8</td>
        <td>Ячейка 9</td>
    </tr>
</table>

```

Так код будет выглядеть в браузере:

Ячейка 1	Ячейка 2	Ячейка 3
Ячейка 4	Ячейка 5	Ячейка 6
Ячейка 7	Ячейка 8	Ячейка 9

Значение bottom

А теперь **по нижнему краю**:

```

td {
    vertical-align: bottom;
}

```

```

}
<table border="1" height="200">
    <tr>
        <td>Ячейка 1</td>
        <td>Ячейка 2</td>
        <td>Ячейка 3</td>
    </tr>
    <tr>
        <td>Ячейка 4</td>
        <td>Ячейка 5</td>
        <td>Ячейка 6</td>
    </tr>
    <tr>
        <td>Ячейка 7</td>
        <td>Ячейка 8</td>
        <td>Ячейка 9</td>
    </tr>
</table>

```

Так код будет выглядеть в браузере:

Ячейка 1	Ячейка 2	Ячейка 3
Ячейка 4	Ячейка 5	Ячейка 6
Ячейка 7	Ячейка 8	Ячейка 9

```

td {
    vertical-align: middle;
}
<table border="1" height="200">
    <tr>
        <td>Ячейка 1</td>
        <td>Ячейка 2</td>
        <td>Ячейка 3</td>
    </tr>
    <tr>
        <td>Ячейка 4</td>
        <td>Ячейка 5</td>
        <td>Ячейка 6</td>
    </tr>
    <tr>
        <td>Ячейка 7</td>
        <td>Ячейка 8</td>
        <td>Ячейка 9</td>
    </tr>
</table>

```

Так код будет выглядеть в браузере:

Ячейка 1	Ячейка 2	Ячейка 3
Ячейка 4	Ячейка 5	Ячейка 6
Ячейка 7	Ячейка 8	Ячейка 9

Практическая работа 2

Селекторы

Задачи на основные CSS свойства

Основы CSS мы разобрали и вы конечно же все сами попробовали вставить в редактор и посмотреть. теперь перейдем к упражнениям

Задачи для решения

На цвета

Для решения задач данного блока вам понадобятся следующие CSS свойства: [color](#).

Сделайте все абзацы `<p>` красного цвета.

Сделайте все `<h1>` зеленого цвета.

Сделайте все `<h2>` голубого цвета.

Сделайте все `<h3>` оранжевого цвета.

На style

Для решения задач данного блока вам понадобятся следующие HTML атрибуты: [style](#).

Сделайте первый на странице абзац `<p>` зеленого цвета.

Сделайте второй на странице абзац `<p>` красного цвета.

На ширину и высоту

Для решения задач данного блока вам понадобятся следующие CSS свойства: [width](#), [height](#).

Сделайте все абзацы `<h2>` шириной 300px.

Сделайте все таблицы `<table>` шириной 400px, высотой 200px.

На выравнивание

Для решения задач данного блока вам понадобятся следующие CSS свойства: [text-align](#).

Поставьте все `<h1>` по центру.

Поставьте все `<h2>` по правому краю.

Сделайте так, чтобы текст в абзацах `<p>` был выровнен одновременно и по правому и по левому краю.

Сделайте так, чтобы во втором абзаце `<p>` текст был выровнен по центру.

Поставьте все `<th>` по левому краю.

Поставьте все `<td>` по центру.

На жирность

Для решения задач данного блока вам понадобятся следующие CSS свойства: [font-weight](#).

Сделайте все `<td>` жирным.

Сделайте `<h1>` нежирным.

Сделайте одновременно `<th>`, `<h1>` и `<h2>` нежирным.

На курсив

Для решения задач данного блока вам понадобятся следующие CSS свойства: [font-style](#).

Сделайте все `<h2>` курсивом.

Сделайте все абзацы `<p>` курсивом, а первый абзац - нет.

На размер шрифта

Для решения задач данного блока вам понадобятся следующие CSS свойства: [font-size](#).

Сделайте все `<h2>` 20px.

Сделайте все абзацы `<p>` 15px.

На семейство

Для решения задач данного блока вам понадобятся следующие CSS свойства: [font-family](#).

Сделайте для абзацев `<p>` шрифт Arial.

Сделайте для `<h2>` шрифт Times New Roman.

Сделайте для `<h3>` любой шрифт без засечек.

На межстрочный интервал

Для решения задач данного блока вам понадобятся следующие CSS свойства: [line-height](#).

Сделайте межстрочный интервал для абзацев `<p>` в 30px.

На свойство-сокращение font

Для решения задач данного блока вам понадобятся следующие CSS свойства: [font](#).

Закомментируйте все стили для абзацев.

Для `<p>` сделайте шрифт Arial, 16 пикселей, курсив, жирный, межстрочный интервал в 30px.

Для `<h1>` сделайте следующий шрифт: нежирный, 20 пикселей, Verdana.

На красную строку

Для решения задач данного блока вам понадобятся следующие CSS свойства: [text-indent](#).

Сделайте красную строку в абзацах 30px.

Для второго абзаца `<p>` уберите красную строку.

На вертикальное выравнивание

Для решения задач данного блока вам понадобятся следующие CSS свойства: [vertical-align](#).

Поставьте текст в таблице `<table>` по верхнему краю по вертикали.

Поставьте текст в `<th>` по центру по вертикали.

Повторите страницы по образцу

Повторите страницу по данному по образцу:

Что такое CMS

CMS - «система управления контентом» (**движок**) – написанная PHP-программистами основа для сайта, с помощью которой вы сможете управлять сайтом (добавлять контент, менять пункты меню и т.п.) не зная HTML и CSS.

Однако, для того чтобы сделать сайт с помощью **CMS** потребуются услуги и программиста, и дизайнера, и верстальщика. И капиталовложения.

Какие бывают cms

Бывают различные системы управления контентом: для интернет-магазинов, для блогов, для форумов и т.д.

Примеры cms

Примеры популярных CMS: Joomla, WordPress (для блогов), PhpBB (для форумов).

CMS-ки бывают *платные* и *бесплатные*.

Текст этого задания

Что такое CMS

CMS - «система управления контентом» (**движок**) – написанная PHP-программистами основа для сайта, с помощью которой вы сможете управлять сайтом (добавлять контент, менять пункты меню и т.п.) не зная HTML и CSS.

Однако, для того чтобы сделать сайт с помощью **CMS** *потребуется* *услуги* и программиста, и дизайнера, и верстальщика. И капиталовложения.

Какие бывают cms

Бывают различные системы управления контентом: для интернет-магазинов, для блогов, для форумов и т.д.

Примеры cms

Примеры популярных CMS: Joomla, WordPress (для блогов), PhpBB (для форумов).

CMS-ки бывают *платные* и *бесплатные*.

Повторите страницу по данному образцу:

Что нужно знать, чтобы делать сайты

1. HTML
2. CSS
3. PHP
4. SQL
5. JavaScript
6. jQuery
7. Flash
8. SEO

PHP и JavaScript

Языки программирования **PHP** и **JavaScript** позволяют сделать сайт динамичным, то есть реагирующим на действия пользователя. Например, можно сделать красивую выпадающую менюшку или слайдер

Виды скриптов

Для этого пишутся скрипты (англ. *script* - «сценарий») - программы, позволяющие реагировать на действия пользователя. Скрипты бывают двух видов:

- те, которые выполняются на сервере, а результат их выполнения приходит в браузер к пользователю уже в готовом виде. Это скрипты, написанные на языке **PHP**. На нем пишутся **CMS-ки** – системы управления контентом.
- те, которые выполняются прямо в браузере пользователя. Это скрипты, написанные на языке **JavaScript**. Они чаще всего используются для, того чтобы сделать страницу более удобной и красивой.

Текст этого задания

Что нужно знать, чтобы делать сайты

1. HTML
2. CSS
3. PHP
4. SQL
5. JavaScript
6. jQuery
7. Flash
8. SEO

PHP и JavaScript

Языки программирования **PHP** и **JavaScript** позволяют сделать сайт динамичным, то есть реагирующим на действия пользователя. Например, можно сделать красивую выпадающую менюшку или слайдер

Виды скриптов

Для этого пишутся скрипты (англ. **script** - «сценарий») - программы, позволяющие реагировать на действия пользователя. Скрипты бывают двух видов:

- *те, которые выполняются на сервере, а результат их выполнения приходит в браузер к пользователю уже в готовом виде. Это скрипты, написанные на языке **PHP**. На нем пишутся **CMS-ки** – системы управления контентом.*
- *те, которые выполняются прямо в браузере пользователя. Это скрипты, написанные на языке **JavaScript**. Они чаще всего используются для, того чтобы сделать страницу более удобной и красивой.*